Reporte de simulación y pruebas

Firmware con IA en el borde para la limpieza electrodinámica de paneles solares

Equipo Spark X · AI-SED 2026 · 6 de noviembre de 2026

Resumen. Este reporte documenta cómo se verificó el firmware de las tarjetas del arreglo (STM32F411RC) y el modelo de IA que lleva dentro. Cada prueba se ejecutó de nuevo el 6 de noviembre de 2026 a partir del código fuente entregado, y los registros completos están en la carpeta `07_Resultados_pruebas`. Resultados principales: el firmware compila sin advertencias y ocupa 47.9 KB de flash; las tablas del árbol de decisión dentro de los 14 archivos `.hex` son idénticas al modelo entrenado; la versión en C reproduce al modelo de Python sin una sola diferencia en 3 153 600 salidas; la máquina de estados de la limpieza pasa sus 8 casos de seguridad; y la red de 6 tarjetas funcionó 28 días simulados sin ninguna activación de alta tensión con agua. La ejecución en Proteus está preparada pero todavía no se corre: el banco de 2 tarjetas ya está armado en Proteus 9.2, que sí incluye el modelo de simulación del STM32F411RC, y falta cargar los `.hex`.

1. Alcance y método

El sistema tiene tres capas, y cada una se probó con un método distinto:

- **Modelo de IA** (árbol de decisión de 125 nodos que estima la probabilidad de agua en el vidrio en las próximas 8 h). Se comprobó que el modelo exportado a C es el mismo que se evaluó en Python, y que el que viaja en cada `.hex` es ese mismo.
- **Núcleo de control** (`sx_controlador.c`: seguridad determinista, características, IA y política). Se ejecutó en la PC dentro de la simulación física del panel, comparado paso a paso contra la referencia de Python (prueba *software-in-the-loop*, SIL).
- **Firmware de la tarjeta** (red de un hilo, árbitro de turnos, ciclo EDS, telemetría). Se compiló el mismo código que corre en el STM32 junto con un HAL que simula sensores, alta tensión y enlaces bit por bit. Así se corrieron 6 tarjetas durante 28 días.

La Tabla 1 resume todas las pruebas. La última fila es la simulación en Proteus, que ejecuta el binario real de ARM sobre el modelo del microcontrolador.

Tabla 1: Matriz de pruebas

#	Prueba	Criterio de aceptación	Resultado
P1	Compilación para STM32 (producción, demo F411, demo F401, demo completo)	0 errores y 0 advertencias con `-Wall -Wextra`	**Pasa**
P2	Uso de memoria	Flash $<$ 128 KB y RAM $<$ 128 KB con pila	**Pasa** (47.9 KB / 17.4 KB)
P3	Integridad del núcleo validado	SHA-256 idéntico al de la validación	**Pasa**
P4	Modelo dentro de cada `.hex`	Tablas idénticas al modelo entrenado; 0 diferencias en 50 000 inferencias	**Pasa** (14/14 archivos)
P5	Equivalencia Python $\leftrightarrow$ C (SIL)	0 diferencias en todas las salidas y lazo cerrado idéntico	**Pasa** (0 de 3 153 600)
P6	Máquina de estados EDS (8 casos)	Código de retorno esperado y alta tensión descargada al terminar	**Pasa** (8/8)
P7	Tolerancia del enlace a la diferencia de reloj	100 % de tramas en $\pm$2 % (peor caso del HSI)	**Pasa** ($\pm$4 %)
P8	Red de 6 tarjetas, 28 días, PCB actual	0 ms de alta tensión con agua; $\leq$1 limpieza simultánea; 0 CRC malos	**Pasa**
P9	Red de 6 tarjetas, 28 días, hardware completo	Igual que P8 y alertas de mantenimiento en las tarjetas con falla	**Pasa**
P10	Escenario del demo (lo que debe verse en Proteus)	Limpieza antes del rocío y bloqueo mientras hay agua	**Pasa** (anticipa 9.5 h)
P11	Simulación en Proteus 9.2 (binario ARM real)	Terminal muestra `amb/est/turno/fin`; fases a 50 Hz en el osciloscopio	**Pendiente**

2. Compilación y recursos (P1–P3)

Se compiló con `arm-none-eabi-gcc` para Cortex-M4F con FPU de precisión simple (`-mcpu=cortex-m4`, `-mfpu=fpv4-sp-d16`, `-mfloat-abi=hard`) y `-ffp-contract=off`. Esta última evita que el compilador fusione multiplicaciones y sumas, para que el micro redondee igual que la simulación validada. Las 14 imágenes se generaron sin errores ni advertencias. La Tabla 2 desglosa el firmware de producción.

Tabla 2: Memoria del firmware de producción (`sparkx_nodo.elf`)

Módulo	Flash (B)	RAM (B)
`sx_nodo.c` (aplicación, red, árbitro)	7 670	–
`sx_controlador.c` (seguridad + IA + política)	5 884	–
de lo cual, tablas del árbol	1 750	–
`sx_hal_stm32.c` (registros del STM32)	2 537	5 480
`sx_red.c`, `sx_uart1h.c`, `sx_eds.c`, `sx_adquisicion.c`	2 946	–
Biblioteca C (`printf` con flotantes, `expf`, ...)	≈28 700	–
Total (`.text + .data`)	47 936 (37 % de 128 KB)	9 856 + 8 192 de pila

El núcleo `nucleo_v2/` (controlador, configuración y modelo) se verificó contra las sumas SHA-256 registradas cuando se hizo la validación (`make nucleo`): los 4 archivos coinciden. Por tanto, el código que corre en el micro es el mismo que se evaluó en el paper.

3. El modelo dentro del `.hex` (P4)

El modelo no se instala por separado: son cinco arreglos constantes en flash (`SX_ARBOL_FEAT`, `THR`, `IZQ`, `DER` y `PROB`). Para demostrar que el archivo que se carga en Proteus o en la tarjeta lleva el modelo correcto, el script `python/verificar_hex.py` hace lo siguiente con cada `.hex`:

1. Lee el archivo en formato Intel HEX, valida el *checksum* de cada línea y reconstruye la flash byte por byte.
2. Obtiene del `.elf` la dirección de cada tabla (por ejemplo, `SX_ARBOL_THR` en `0x0800AB60`, 500 B) y la extrae de la imagen.
3. Compara cada tabla contra `resultados/modelo_rocio.json`: índices exactos y umbrales y probabilidades en `float32` bit a bit.
4. Recorre el árbol extraído con 50 000 vectores aleatorios que cubren el rango físico de las 10 características, y compara la probabilidad contra el árbol de referencia.

Resultado: las tablas son idénticas en los 10 archivos para el F411RC y en los 4 para el F401RC, con 0 diferencias en 700 000 inferencias. Ocupan 1 750 B: 125 nodos × (3 índices de 2 B + 2 flotantes de 4 B).

4. Equivalencia Python ↔ C (P5)

`python/sil_validacion.py` compila el núcleo en C como biblioteca y lo ejecuta en paralelo con el controlador de referencia en Python. Las corridas cubren 6 años de 35 040 pasos con distintas semillas de clima, políticas y fallas inyectadas: conector degradado, punto caliente, sensor congelado y lodo. En cada paso compara las 15 salidas: bits de seguridad, características, probabilidad de la IA, decisión, alertas y pronósticos.

- Paso a paso: 210 240 pasos, 3 153 600 salidas comparadas, **0 diferencias**.
- Lazo cerrado (el C decide y la física responde), 4 años: pérdida por suciedad, ciclos EDS (226, 276, 223, 182), lavados, activaciones inseguras (0) y energía neta **idénticos** a los de Python.

5. Pruebas unitarias del firmware (P6, P7)

5.1 Ciclo de limpieza EDS

`host/test_eds.c` ejecuta la máquina de estados del ciclo EDS (arranque del flyback, verificación del 70 % de 1.2 kV a los 50 ms, 3 fases a 50 Hz y descarga a <50 V) contra un modelo de la alta tensión. Se corren 4 escenarios, con y sin medición de corriente de alta tensión.

Tabla 3: Casos de la máquina de estados EDS

Escenario	Configuración	rc esperado	rc obtenido	Estado final
Vidrio seco	sin / con I_{AT}	0 / 0	0 / 0	AT 0 V, flyback y fases apagados
Agua al arrancar (AT no sube)	sin / con I_{AT}	−1 / −1	−1 / −1	ídem
Rocío a media limpieza (AT cae)	sin / con I_{AT}	−2 / −2	−2 / −2	ídem
Fuga sin caída de AT	sin / con I_{AT}	0 / −2	0 / −2	ídem

El último caso documenta una limitación conocida. Sin sensor de corriente, que es el caso de la PCB actual, una fuga que no hace caer la tensión no se detecta durante la limpieza. Por eso el firmware exige 1 h de vidrio claramente seco antes de limpiar (bit `0x40`).

5.2 Tolerancia del enlace de un hilo

Las tarjetas usan el oscilador interno (HSI, ±1 %) si no hay cristal. `host/test_deriva.c` envía 200 tramas de 40 B entre dos UART por software con relojes desplazados. La recepción es perfecta entre −4 % y +4 % (Fig. 1): el margen es 2 veces el peor caso entre dos tarjetas.

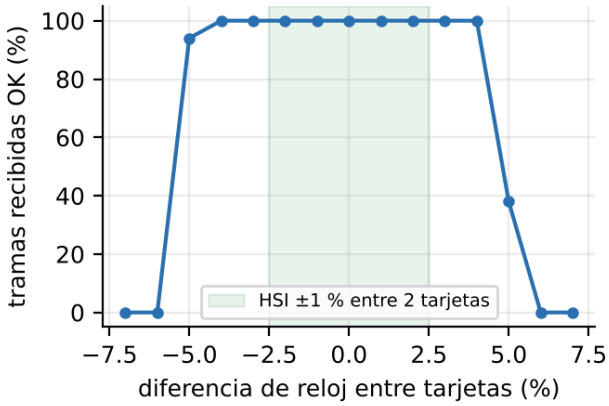


Figura 1: Tramas recibidas contra la diferencia de reloj entre tarjetas (4800 baudios, sobremuestreo ×8).

6. Red de 6 tarjetas durante 28 días (P8, P9)

`host/sim_red.c` enlaza seis copias del firmware real (arreglo de 2×3, nodo 1 coordinador) y simula cada bit de los enlaces. Cada tarjeta lee sus propios datos: mismo clima, pero sensores, polvo y fallas propios. El escenario incluye tres eventos de red:

- día 12, 03:00–09:00: **se apaga el coordinador**. Los nodos bloquearon la alta tensión a los 2 pasos, al expirar el dato de ambiente, y se recuperaron solos al volver;
- día 16, 12:00: **se corta el cable 1–2**. La red rehízo la ruta: los nodos 2 y 3 llegaron al coordinador por 5–4–1 (saltos 3 y 4);
- día 22, 08:00: el operador registra un **lavado** del nodo 2, recibido y confirmado.

Sin mini-celdas, la PCB actual no mide el polvo: limpia una vez al día y, además, cuando la IA anticipa rocío. Por eso hace 53 ciclos en 28 días. Con el hardware completo limpia solo cuando el polvo lo justifica, entre 9 y 13 veces. Las alertas de conector y punto caliente aparecen en las tarjetas a las que se inyectaron esas fallas (3 y 4); sin la carga de prueba ni el NTC, la PCB actual no puede generarlas.

7. Escenario del demo: lo que debe verse en Proteus (P10)

Los `.hex` de demostración leen los sensores de tablas grabadas en flash, generadas de los mismos datos, y aceleran el tiempo: un paso de 15 min dura 4 s. Al revisar las pruebas se encontró que el demo anterior arrancaba

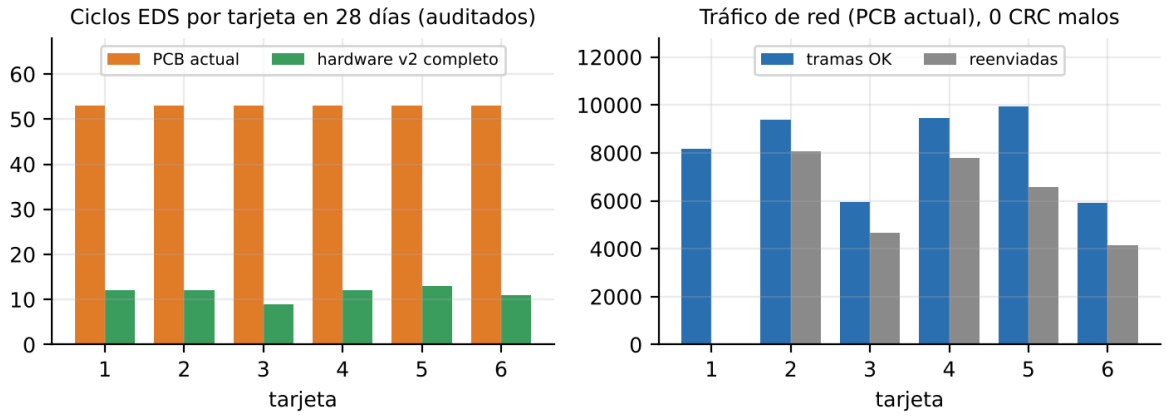


Figura 2: Izquierda: ciclos EDS por tarjeta, contados por el simulador (independiente del contador interno, que se reinicia si la tarjeta se apaga). Derecha: tramas recibidas y reenviadas por tarjeta.

Tabla 4: Auditoría de la red en 28 días (2 688 pasos de 15 min, 826 millones de ticks de enlace)

Indicador	PCB actual	Hardware completo
Milisegundos de alta tensión con agua en el vidrio	0	0
Máximo de limpiezas simultáneas (límite = 1)	1	1
Limpiezas abortadas / iniciadas con ambiente vencido	0 / 0	0 / 0
Tramas con CRC incorrecto / colisiones / errores de marco	0 / 0 / 0	0 / 0 / 0
Ciclos EDS por tarjeta	53	9 a 13
Alertas de mantenimiento	EDS en nodo 2	EDS (2), conector (3), punto caliente (4)

el día 8 (`SX_K0=768`), y ese día no hay rocío en los datos, así que la anticipación de la IA no se habría visto. Se cambió el arranque al día 15 a las 18:00 (`SX_K0=1512`). Se recompilaron los 12 `.hex` de demo y se reprodujo el escenario con la simulación de red, usando el mismo paso de 4 s (Fig. 3).

Secuencia esperada (tiempo de simulación entre paréntesis):

1. 18:00–19:00 (0–16 s): alta tensión bloqueada (`bits=64`, `0x40`) mientras se comprueba 1 h de vidrio seco.
2. 19:45 (28 s): la IA sube a $p = 0.61 \geq 0.60$ y el nodo pide turno; el coordinador concede `turno` a los nodos 1 y 2 y cada uno reporta `fin` con `rc=0` y `hv_max`≈1250 V. Es la **limpieza antes del rocío**, 9.5 h antes del agua.
3. 00:30 (104 s): el margen cae bajo 4 °C y la seguridad bloquea la alta tensión (`0x40`); a la 01:30 se suma `0x02` (margen < 2 °C).
4. 05:15–08:00 (180–224 s): hay agua en el vidrio; la alta tensión sigue bloqueada todo el tiempo (0 activaciones con agua).
5. 09:00 (240 s): el vidrio ya está seco y se hace la limpieza diaria.

Limitación del modo demo. Con pasos de 4 s, el árbitro solo alcanza a dar turno a las tarjetas a un salto del coordinador. En la simulación de 6 tarjetas con paso de 4 s, las tarjetas a 2 o más saltos agotaron su espera (60 % del paso) sin turno. Con el paso real de 15 min, y en la simulación de 8 s, todas las tarjetas limpian. Por eso el banco de Proteus usa 2 tarjetas vecinas.

8. Simulación en Proteus (P11)

8.1 Banco de prueba

Se armó el proyecto `SparkX_SIM_IA.pdsprj` en Proteus Professional 9.2 SP1. Aunque el asistente de proyectos de firmware solo ofrece el STM32F401RC, la biblioteca `CM4_STM32` sí incluye el **STM32F411RC con modelo de simulación** (`STM32F411RC.MDF`). Así, la simulación usa el mismo micro que la PCB (U9), con el `.hex` del F411RC. Para versiones de Proteus sin ese modelo se entrega además el demo compilado para el STM32F401RC, que solo cambia el tamaño de la RAM (64 KB).

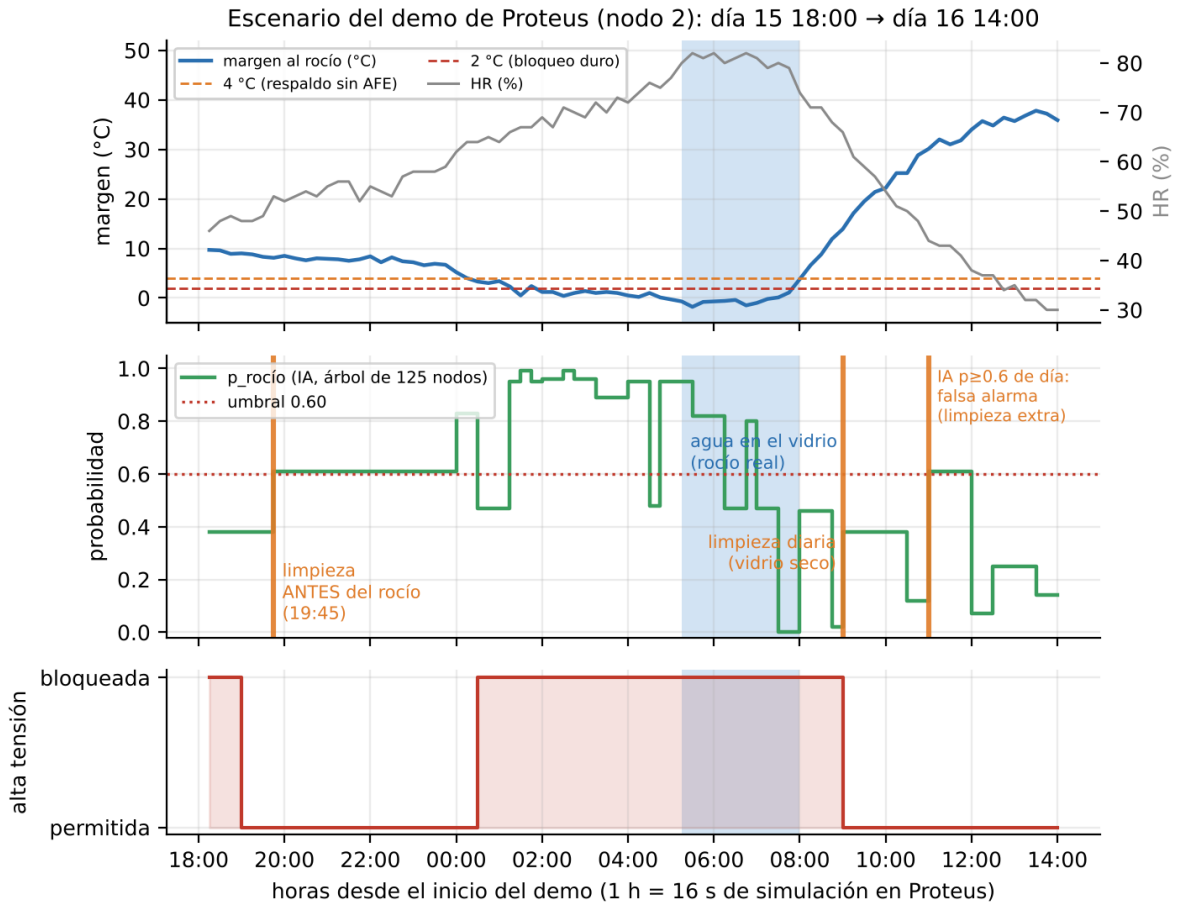


Figura 3: Escenario del demo para el nodo 2. Sombreado azul: agua real en el vidrio. Líneas naranjas: limpiezas concedidas por el coordinador.

8.2 Estado

Ya están colocados y cableados U1, U2, las dos terminales virtuales y el osciloscopio de U1 (Fig. 4). Falta lo siguiente:

1. Cargar `demo_nodo1.hex` y `demo_nodo2.hex` en *Program File*. Los archivos todavía no están en la computadora donde corre Proteus.
2. Cablear el enlace PB13–PC10 con su resistencia de 4.7 kΩ y la terminal de U2.
3. Correr el escenario y comparar la terminal contra la Sección 7.

Durante el armado, Proteus 9.2 SP1 se cerró una vez con un error interno (*access violation* en `APPFRAME.DLL`) al navegar la vista. El proyecto ya estaba guardado y no se perdió nada. Labcenter publicó la versión 9.2 SP3, que conviene instalar antes de la corrida larga.

8.3 Criterios de aceptación

- La terminal 1 imprime el encabezado `=== Spark-X red | nodo 1 (COORDINADOR) ...` y una línea `amb` cada 4 s.
- Llegan líneas `est` de los nodos 1 y 2 con `saltos` = 0 y 1, sin `error`.
- Alrededor de los 28 s aparecen `turno` y `fin` con `rc=0` para los nodos 1 y 2. En el osciloscopio, PB4 sube y PA2–PA4 conmutan en secuencia de 6 pasos a 50 Hz durante unos 2 s.
- Entre 104 s y 240 s, `bits` $\neq 0$ en las líneas `est` y no aparece ningún `turno`.

Tabla 5: Conexiones del banco (2 tarjetas vecinas)

Elemento	Conexión	Propósito
U1 STM32F411RC	*Program File* = `demo_nodo1.hex`; cristal 8 MHz	coordinador (nodo 1)
U2 STM32F411RC	*Program File* = `demo_nodo2.hex`; cristal 8 MHz	nodo 2
Terminal virtual 1	RXD ← U1 PA9, TXD → U1 PA10; 115200 8N1	telemetría JSON y comandos
Terminal virtual 2	RXD ← U2 PA9; 115200 8N1	consola del nodo 2
Enlace de un hilo	U1 PB13 (RIGHT_PANEL) ↔ U2 PC10 (LEFT_PANEL), 4.7 kΩ a VDD	red entre tarjetas
Osciloscopio	A = U1 PB4, B/C/D = U1 PA2/PA3/PA4	flyback y 3 fases del EDS

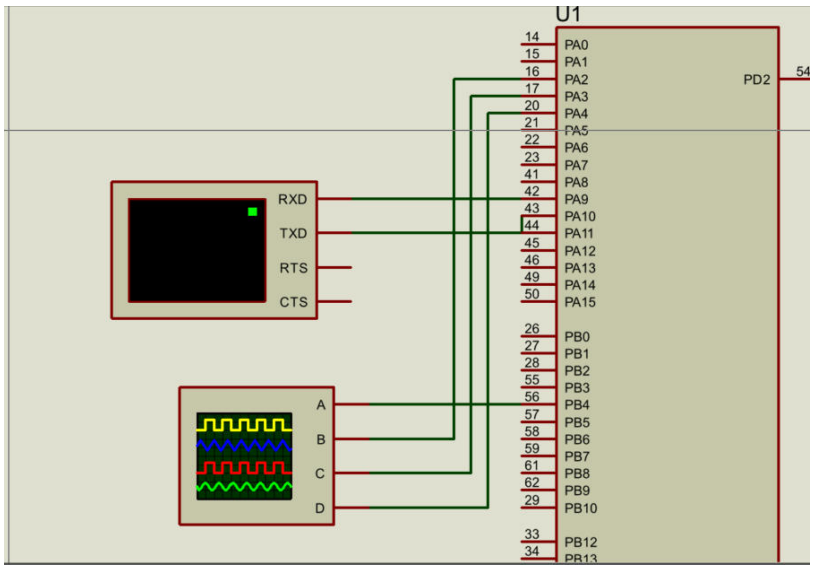


Figura 4: Banco en Proteus 9.2: U1 (coordinador) con la terminal virtual en PA9/PA10 y el osciloscopio en PB4 y PA2–PA4.

9. Hallazgos de esta ronda de pruebas

1. **Escenario del demo sin rocío** (corregido): el arranque se movió del día 8 al día 15, 18:00.
2. **Archivo de la prueba SIL fuera del paquete** (corregido): `sil_wrapper.c` no estaba en la carpeta entregada; se agregó a `nucleo_v2/` y la prueba corre de nuevo desde cero.
3. **Modo demo con muchas tarjetas**: con pasos de 4 s, las tarjetas a 2 o más saltos no alcanzan turno (Sección 7). No afecta la operación real.
4. **Contador de limpiezas tras un reinicio**: el contador interno de cada tarjeta empieza de cero cuando la tarjeta se apaga (el coordinador reporta 34 en lugar de 53 tras el apagado del día 12). Para el mantenimiento conviene guardarlo en flash junto con la configuración.

10. Cómo reproducir

En Linux o WSL, con `gcc`, `gcc-arm-none-eabi`, `libnewlib-arm-none-eabi`, `make` y Python 3 con `numpy` y `scikit-learn`:

```
cd 03_Firmware_codigo/firmware
make stm32 proteus proteus_f401 proteus_completo # P1: 14 archivos .hex
make tamano # P2
make nucleo # P3
make pruebas # P6, P7, P8 y P9 (~35 s)
cd ../python
python3 verificar_hex.py # P4 (F411RC)
python3 verificar_hex.py ../firmware/build/f401 # P4 (F401RC)
python3 sil_validacion.py # P5 (~35 s)
```

```
python3 figuras_simulacion.py # figuras de este reporte
```